Universidad Peruana de Ciencias Aplicadas
Facultad de Ingeniería



Curso: Matemática Computacional

Profesor: Luis Daniel Muñoz Ramos

Sección: 5829

Proyecto: Problema del camino mínimo

Integrantes
Chuquisengo Gomez, Aldighieri Farid

Contenido

1. Introducción **2**

2. Objetivo **3**

3. Fundamento teórico **3**

3.1. Grafos 3

3.1.1. Grafos conexos 4

3.1.2. Grafos ponderados 4

3.2. Matriz de adyacencia 5

3.3. Problema del camino mínimo 6

3.4. Algoritmo de Dijkstra 7

4. Aplicaciones del proyecto **11**

5. Diagrama de flujo **12**

6. Resultados del código **12**

7. Conclusiones **20**

8. Bibliografía **22**

Problema de camino mínimo

1. Introducción

En esencia, el problema del camino mínimo (o camino más corto) consiste en: dado un grafo con pesos en sus aristas, hallar el camino de menor costo entre dos vértices (o desde un origen a todos los otros). Es difícil atribuir un "primer autor" para esta formulación, pues la idea de optimizar rutas ha existido implícitamente desde que los humanos y sistemas de transporte querían minimizar distancias o costos. Como indican algunas revisiones históricas, *"It is difficult to trace the origin of the shortest path problem"* (Cooke et al., 2003). No obstante, su estudio sistemático en teoría de grafos y algoritmos surge en la segunda mitad del siglo XX, en el contexto de ciencias de la computación y optimización. El hito clásico es Edsger W. Dijkstra, quien en 1956 ideó un algoritmo para hallar el camino más corto desde un vértice origen a los demás, en grafos con pesos no negativos :*"The classical shortest path algorithm was discovered by Dijkstra in 1956 and published in 1959"* (Cooke et al., 2003). Publicó el resultado en 1959 en un artículo titulado *"A Note on Two Problems in Connexion with Graphs"*. Según relatos (incluyendo los del propio Dijkstra), la idea inicial le vino en un café, mientras reflexionaba sobre la ruta más corta entre ciudades holandesas: *"the algorithm was conceived in about twenty minutes while Dijkstra was sitting in a café in Amsterdam with his fiancée and trying to solve the problem of the shortest path from Rotterdam to Groningen"* (Cornell University, 2001). Originalmente, lo desarrolló mentalmente en unos 20 minutos.

En este proyecto, se presenta un programa en Python el cual desarrollará problemas de camino mínimo en grafos utilizando el algoritmo de Dijkstra. En esta primera entrega se verá un avance del 50%, con la expectativa de completar el programa en posteriores entregas. Para ejecutar el programa se necesitará del programa Visual Studio Code.

A continuación, se presentará el objetivo del proyecto, seguido del fundamento teórico que sustenta nuestro trabajo. Posteriormente, se describirá la aplicación práctica del proyecto y se proporcionará una descripción detallada del código del programa. Finalmente, se incluirá la coevaluación realizada por los integrantes del equipo, evaluando la participación, calidad del trabajo, colaboración, responsabilidad y cumplimiento de tareas.

2. Objetivo

Desarrollar un programa en Python con estrategias puras como grafos para rutas óptimas y problemas de camino mínimo para los equilibrios de mejora, por medio de una interfaz sencilla e intuitiva.

3. Fundamento teórico

3.1. Grafos

Un grafo es una estructura matemática que se utiliza para modelar relaciones entre objetos. Formalmente, se define como un par ordenado:

$$G = (V, E)$$

donde:

- V es conjunto no vacío de vértices (o nodos),
- E es un conjunto de aristas (o conexiones) que unen pares de vértices.

Las aristas pueden ser dirigidas o no dirigidas, dependiendo de si la relación entre los vértices tiene una dirección específica. Además, en muchos contextos, las aristas pueden tener un valor numérico asociado, conocido como peso, que puede representar costos, distancias o capacidades, convirtiendo al grafo en un grafo ponderado. (Chacón, 2005)

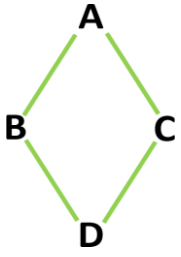
Ejemplo:

Supongamos el conjunto de vértices: $V = \{A, B, C, D\}$.

Y las conexiones entre los vértices se representan mediante el conjunto de aristas:

$$V = \{\{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}\}.$$

Se representaría el grafo como:



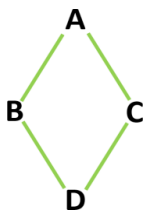
3.1.1. Grafos conexos

Según Martínez (s.f.), un grafo conexo es un grafo no dirigido en el que existe al menos un camino entre cualquier par de vértices. Esta propiedad es fundamental en los problemas de camino mínimo, ya que garantiza que siempre se puede encontrar un recorrido entre los nodos de interés.

Ejemplo:

Consideremos el conjunto de vértices: $V = \{A, B, C, D\}$ y el conjunto de aristas: $E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}\}$.

La representación de su grafo sería:



- Este grafo es **conexo**, porque desde cualquier vértice se puede llegar a cualquier otro. Por ejemplo, de A a D existe el camino $A \rightarrow B \rightarrow D$.
- Si eliminámos, por ejemplo, la arista $\{C, D\}$, seguiriía siendo conexo, pero si eliminamos $\{B, D\}$ y $\{C, D\}$, dejaría de serlo, pues el vértice D quedaría asilado de A.

3.1.2. Grafos ponderados

Según García (2019), un grafo ponderado es un grafo en el que a cada arista se le asigna un valor numérico, conocido como peso. Formalmente, se puede representar como un trío ordenado $G = (V, E, W)$,

donde:

- V es el conjunto de vértices
- E es el conjunto de aristas
- W es una función $W: E \rightarrow R$ que asigna un peso real a cada arista.

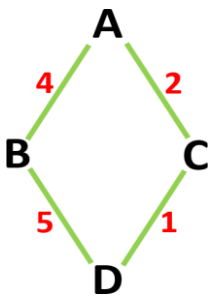
Esta estructura permite modelar situaciones donde las conexiones entre los vértices tienen diferentes costos, distancias o capacidades, siendo fundamental en algoritmos de optimización y análisis de redes.

Ejemplo:

Consideremos el conjunto de vértices $V = \{A, B, C, D\}$ y el conjunto de aristas $E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}\}$. Supongamos que los pesos de las aristas son los siguientes:

- $w(A, B) = 4$
- $w(A, C) = 2$
- $w(B, D) = 5$
- $w(C, D) = 1$

Este grafo se representa como:



3.2. Matriz de adyacencia

Según Lluch (2010), la matriz de adyacencia es una forma de representar un grafo mediante una matriz cuadrada. Si el grafo tiene n vértices, se construye una matriz A de tamaño $n \times n$, donde cada entrada a_{ij} indica la reacción entre los vértices i y j:

$$a_{ij} = \begin{cases} 1, \text{ si existe una arista entre el vértice } i \text{ y } j. \\ 0, \text{ en caso contrario.} \end{cases}$$

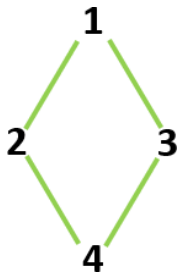
En grafos ponderados, en lugar de un 1 se coloca el peso de la arista. Si el grafo es no dirigido, la matriz es simétrica ($a_{ij} = a_{ji}$).

Ejemplo:

Consideremos un grafo no dirigido con los vértices $V = \{1, 2, 3, 4\}$ y aristas:

$$E = \{(1; 2), (1; 3), (2; 4), (3; 4)\}.$$

Su representación gráfica sería:



Por ende, la matriz de adyacencia correspondiente es:

- La fila 1 muestra: $a_{12} = 1$, $a_{13} = 1$. Es decir, 1 está conectado con 2 y 3.

$$\begin{array}{c} \\ 1 \\ 2 \\ 3 \\ 4 \end{array} \begin{array}{c} \begin{array}{cccc} 1 & 2 & 3 & 4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \end{bmatrix} \end{array}$$

- La fila 2 muestra: $a_{21} = 1$, $a_{24} = 1$. Es decir, 2 está conectado con 1 y 4.
- La fila 3 muestra: $a_{31} = 1$, $a_{34} = 1$. Es decir, 3 está conectado con 1 y 4.
- La fila 4 muestra: $a_{42} = 1$, $a_{43} = 1$. Es decir, 4 está conectado con 2 y 3.

3.3. Problema del camino mínimo

El problema del camino mínimo (SPP, Shortest Path Problem) surge principalmente en el contexto de redes de transporte, donde se busca simular cómo un flujo se transmite desde un centro de producción s hasta un centro destinatario t, atravesando distintos nodos de recepción y redistribución. En este escenario, cada enlace de la red tiene asociada una capacidad o un costo, y el objetivo es identificar la ruta más eficiente que permita completar el recorrido con el menor costo posible. (Muñóz, 2010).

De manera formal, el problema del camino mínimo consiste en: dado un grafo $G = (V, E)$ con pesos $w(u, v)$ para cada arista $(u, v) \in E$, y dados dos vértices s (origen) y t (destino), encontrar un camino de s a t cuya suma de pesos de sus aristas sea la **mínima** entre todos los posibles caminos que conectan s con t. (Muñóz, 2010).

Ejemplo:

Supongamos un grafo con vértices $V = \{A, B, C, D\}$ y aristas ponderadas:

- $A - B$: 3
- $A - C$: 1
- $B - C$: 1
- $B - D$: 5
- $C - D$: 2

Si deseamos encontrar el camino mínimo entre A y D, tenemos 2 opciones:

- El camino $A \rightarrow B \rightarrow D$: cuesta 3 + 5 = 8
- El camino $A \rightarrow C \rightarrow D$: cuesta 1 + 2 = 3

Por tanto, el camino mínimo de A a D es $A \rightarrow C \rightarrow D$ con costo total de 3.

3.4. Algoritmo de Dijkstra

El algoritmo de Dijkstra, propuesto en 1959 por Edsger Dijkstra, es uno de los métodos más utilizados para resolver el problema del camino mínimo en grafos ponderados con pesos no negativos. Según Sánchez y Lozano (2001), el objetivo es encontrar el camino más corto desde un nodo origen hasta todos los demás nodos del grafo, construyendo progresivamente una solución mediante una estrategia voraz.

El procedimiento se basa en asignar al nodo origen una distancia inicial de cero, mientras que los demás tienen una distancia aún no determinada. Luego, en cada iteración, se selecciona el vértice con la menor distancia provisional y se actualizan las distancias de sus vértices adyacentes si se encuentra un camino más corto. Este proceso se repite hasta que todos los nodos han sido

procesados (Sánchez y Lozano, 2001).

La notación: $[d,\ p]_{(k)}$,

se utiliza para representar:

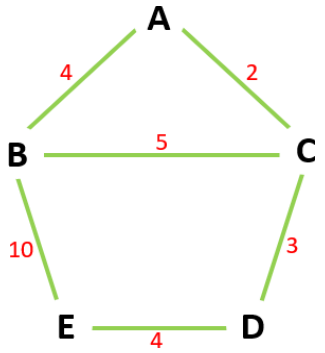
- d: la distancia provisional mínima al nodo
- p: el predecesor desde el cual se alcanza
- (k): la iteración en la que se actualizó dicho valor

Ejemplo:

Sea el grafo con vértices $V = \{A,\ B,\ C,\ D,\ E\}$ y aristas ponderadas:

- $A\ -\ B$: 4
- $A\ -\ C$: 2
- $B\ -\ C$: 5
- $B\ -\ E$: 10
- $C\ -\ D$: 3
- $E\ -\ D$: 4

La representación gráfica sería:



Aplicando el algoritmo desde el nodo A:

- **Estado inicial (iteración 0)**
  - $d(A) = [0,\ -\ ]_{(0)}$
  - $d(B) = [\ -\ ,\ -\ ]_{(0)}$
  - $d(C) = [\ -\ ,\ -\ ]_{(0)}$
  - $d(D) = [\ -\ ,\ -\ ]_{(0)}$

- $d(E) = [\ -\ ,\ -\ ]_{(0)}$

- **Iteración 1**

  Buscamos vecinos de A:

  ○ $A \rightarrow B$: alternativa $= 0 + 4 = 4 \Rightarrow$ actualiza $d(B) = [4, A]_{(1)}$

  ○ $A \rightarrow B$: alternativa $= 0 + 4 = 4 \Rightarrow$ actualiza $d(B) = [4, A]_{(1)}$

  Tabla después de la iteración 1:

  - $d(A) = [0,\ -\ ]_{(0)}$
  - $d(B) = [4\ ,\ A\ ]_{(1)}$
  - $d(C) = [2\ ,\ A\ ]_{(1)}$
  - $d(D) = [\ -\ ,\ -\ ]_{(0)}$
  - $d(E) = [\ -\ ,\ -\ ]_{(0)}$

- **Iteración 2**

  Elegimos el vértice con menor distancia: $C\ (d = 2)$

  Buscamos vecinos de C:

  ○ $C \rightarrow A$: ya visitado – ignorar

  ○ $C \rightarrow B$: alternativa $= 2 + 5 = 7 \Rightarrow$ no se actualiza porque es mayor a la distancia anterior.

  ○ $C \rightarrow D$: alternativa $= 2 + 3 = 5 \Rightarrow$ actualiza $d(D) = [5, C]_{(2)}$

  Tabla después de la iteración 2:

  - $d(A) = [0,\ -\ ]_{(0)}$
  - $d(B) = [4\ ,\ A\ ]_{(1)}$
  - $d(C) = [2\ ,\ A\ ]_{(1)}$
  - $d(D) = [5\ ,\ C\ ]_{(2)}$
  - $d(E) = [\ -\ ,\ -\ ]_{(0)}$

- **Iteración 3**

  Elegimos el vértice con menor distancia: $B\ (d = 4)$

  Buscamos vecinos de B:

- $B \rightarrow A$: ya visitado – ignorar
- $B \rightarrow C$: ya visitado – ignorar
- $B \rightarrow E$: alternativa $= 4 + 10 = 14 \Rightarrow$ actualiza $d(E) = [14, B]_{(3)}$

Tabla después de la iteración 3:

- $d(A) = [0, -]_{(0)}$
- $d(B) = [4, A]_{(1)}$
- $d(C) = [2, A]_{(1)}$
- $d(D) = [5, C]_{(2)}$
- $d(E) = [14, B]_{(3)}$

- **Iteración 4**

Elegimos el vértice con menor distancia: D $(d = 5)$

Buscamos vecinos de D:

- $D \rightarrow C$: ya visitado – ignorar
- $D \rightarrow E$: alternativa $= 5 + 4 = 9 \Rightarrow$ se actualiza $d(E) = [9, D]$ porque es menor a la distancia anterior.

Tabla después de la iteración 4:

- $d(A) = [0, -]_{(0)}$
- $d(B) = [4, A]_{(1)}$
- $d(C) = [2, A]_{(1)}$
- $d(D) = [5, C]_{(2)}$
- $d(E) = [9, D]_{(4)}$

- **Iteración 5**

Elegimos el único vértice no visitado: E $(d = 9)$

Todos los vecinos de E ya fueron visitados.

Tabla final:

- $d(A) = [0, -]_{(0)}$
- $d(B) = [4, A]_{(1)}$

- $d(C) = [2\ ,\ A\ ]_{(1)}$
- $d(D) = [5\ ,C\ ]_{(2)}$
- $d(E) = [9\ ,\ D\ ]_{(4)}$

Después de aplicar el algoritmo de Dijkstra, encontramos el camino mínimo $A \rightarrow E$: $A \rightarrow C \rightarrow D \rightarrow E$.

Con un costo total = 2 + 3 + 4 = 9.

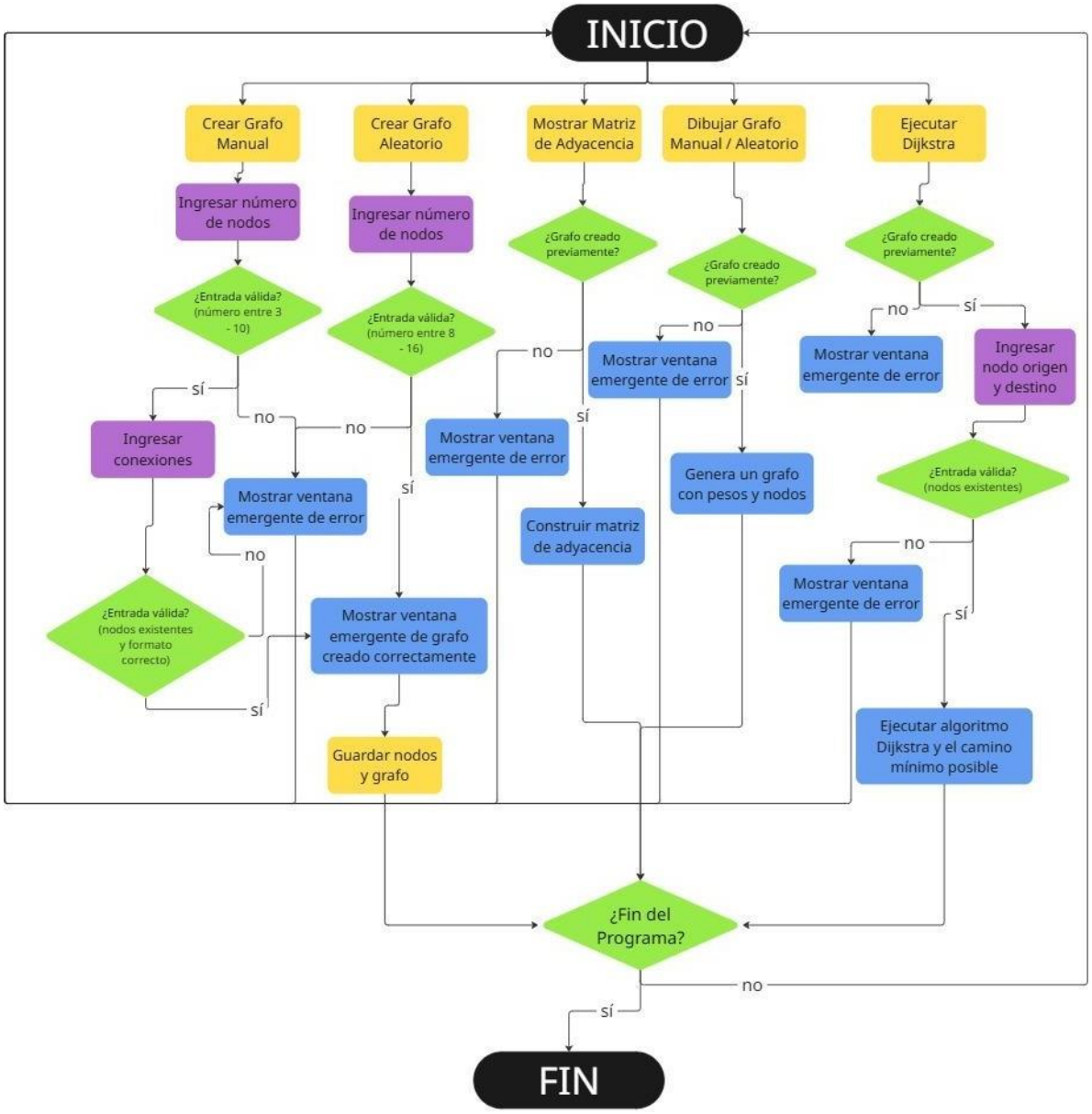
4. Aplicaciones del proyecto

El problema del camino mínimo y el algoritmo de Dijkstra tienen una amplia variedad de aplicaciones en el mundo real, ya que permiten optimizar recorridos, costos y tiempos en diferentes contextos. Entre las principales aplicaciones destacan:

- Redes de transporte: se utiliza para encontrar las rutas más cortas en carreteras, sistemas de metro, líneas de autobuses y logística de distribución. Por ejemplo, determinar el recorrido más rápido entre dos ciudades o calcular la ruta óptima para vehículos de reparto.
- Redes de telecomunicaciones: permite identificar los caminos más eficientes en la transmisión de datos, optimizando el uso del ancho de banda y reduciendo la latencia en redes de internet y telefonía.
- Sistemas de navegación GPS: los dispositivos de posicionamiento global emplean algoritmos de camino mínimo para guiar a los usuarios hacia su destino en el menor tiempo de distancia posible, considerando mapas representados como grafos ponderados.
- Optimización de recursos en logística y producción: ayuda a reducir costos de transporte, minimizar tiempos de entrega y planificar cadenas de suministro eficientes.
- Inteligencia artificial y videojuegos: en entornos virtuales, los personajes no jugadores (NPCs) utilizan algoritmos de camino mínimo para desplazarse de un punto a otro de manera realista y eficiente.

En conclusión, el desarrollo de este proyecto no solo tiene valor académico, sino que también refleja cómo los algoritmos basados en grafos son fundamentales en tecnologías que usamos a diario, desde un simple GPS hasta complejas redes de comunicación y transporte.

5. Diagrama de flujo



6. Resultados del código

- **Bibliotecas**

Tkinter: Se utiliza para la creación de interfaces

Random: Genera números aleatorios

Matplotlib: Librería de graficación 2D

Network: Librería especializada en la creación y manipulación de grafos

Figura 1

```
import tkinter as tk
from tkinter import ttk, messagebox, simpledialog
import random
import matplotlib.pyplot as plt
import networkx as nx
```

Nota. Captura de pantalla de las librerías importas para el código.

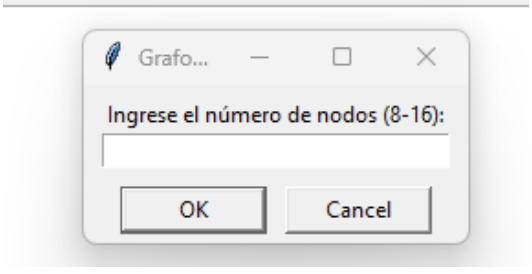
- **Creación del grafo de forma aleatoria:** Se genera una grafo dirigido aleatorio con n nodos, etiquetados de 'A' en adelante. Se establece una probabilidad del 40% para que exista una arista entre cualquier par de nodos, asignándole un peso aleatorio entre 1 y 9.

Figura 2

```
def crear_grafoAleatorio(n):
    lista_nodos = [chr(ord('A') + i) for i in range(n)]
    grafo = [[] for x in range(n)]
    for i in range(n):
        for j in range(n):
            if i != j and random.random() < 0.4:
                peso = random.randint(1, 9)
                grafo[i].append((j, peso))
    return lista_nodos, grafo
```

Nota. Captura de pantalla del código de la función. Devuelve los nodos y el grafo.

Figura 3



Nota. Se le pide el número de nodos al usuario para crear el grafo.

- **Creación del grafo de forma manual:** Se permite la creación manual de un grafo dirigido con n nodos, pidiendo al usuario origen, destino,

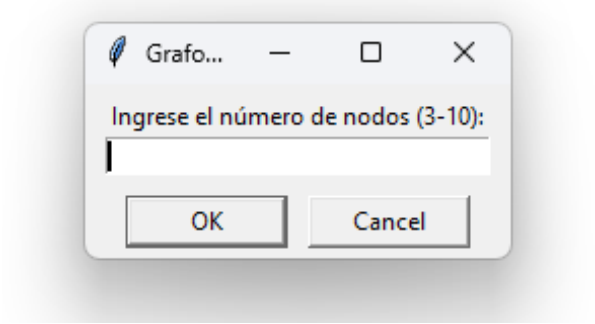
peso. Valida que los nodos existan y añade la arista y su peso a la lista de adyacencia hasta que el usuario escriba ‘fin’ o ‘cancele’.

Figura 4

```python
def crear_grafoManual(n, text_widget):
    lista_nodos = [chr(ord('A') + i) for i in range(n)]
    grafo = [[] for x in range(n)]
    text_widget.insert(tk.END, "Ingrese conexiones entre nodos en formato: origen,destino,peso. (escriba 'fin' para terminar)\n")
    while True:
        entrada = simpledialog.askstring("Grafo Manual", "Conexión: ")
        if entrada is None or entrada.lower() == 'fin':
            break
        try:
            origen, destino, peso = entrada.split(',')
            origen = origen.strip().upper()
            destino = destino.strip().upper()
            peso = int(peso.strip())
            if origen in lista_nodos and destino in lista_nodos:
                i = lista_nodos.index(origen)
                j = lista_nodos.index(destino)
                grafo[i].append((j, peso))
            else:
                text_widget.insert(tk.END, "Nodos inválidos.\n")
        except:
            text_widget.insert(tk.END, "Formato incorrecto. Use: A,B,3\n")
        text_widget.update()
    return lista_nodos, grafo
```

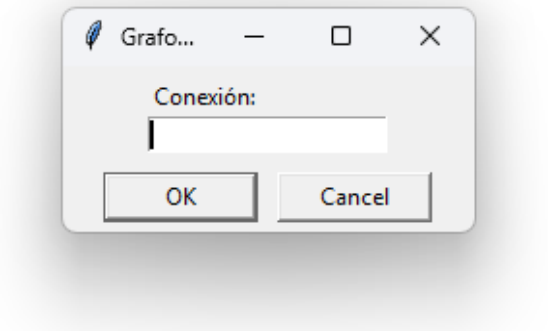
Nota. Captura de pantalla del código de la función. Devuelve los nodos y el grafo.

Figura 5



Nota. Se le pide el número de nodos al usuario para crear el grafo.

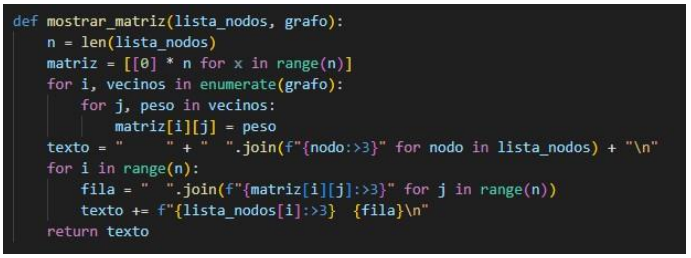
Figura 6



Nota. Se le pide al usuario ingresar las conexiones.

- **Mostrar la matriz de adyacencia:** Se muestra la matriz que se genera a partir del grafo creado anteriormente

Figura 7



```
def mostrar_matriz(lista_nodos, grafo):
    n = len(lista_nodos)
    matriz = [[0] * n for x in range(n)]
    for i, vecinos in enumerate(grafo):
        for j, peso in vecinos:
            matriz[i][j] = peso
    texto = "     " + "  ".join(f"{nodo:>3}" for nodo in lista_nodos) + "\n"
    for i in range(n):
        fila = "  ".join(f"{matriz[i][j]:>3}" for j in range(n))
        texto += f"{lista_nodos[i]:>3}  {fila}\n"
    return texto
```

Nota. Captura de pantalla del código de la función. Devuelve la matriz de adyacencia.

Figura 8

	A	B	C	D	E	F	G	H
A	0	0	0	0	0	0	0	0
B	0	0	4	0	7	9	1	0
C	1	0	0	0	4	3	0	0
D	0	6	9	0	7	0	5	0
E	4	8	7	2	0	0	3	0
F	0	0	8	5	2	0	0	0
G	0	1	7	0	0	0	0	0
H	6	2	0	0	3	7	5	0

Nota. Ejemplo de un grafo generado aleatoriamente.

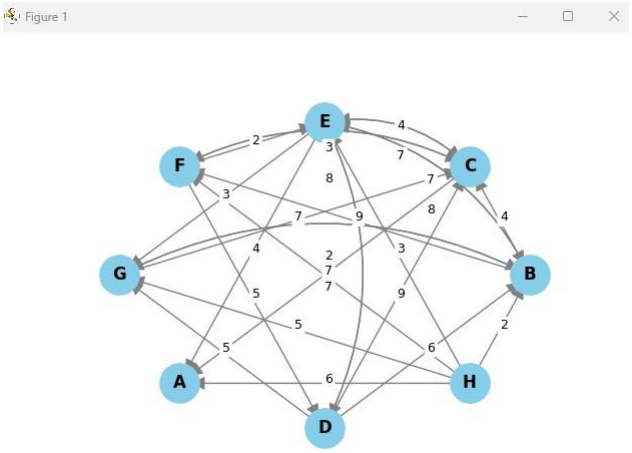
- **Graficación del grafo:** Se grafica el grafo a partir de la matriz de adyacencia

Figura 9

```python
def dibujar_grafo(lista_nodos, grafo, tipo):
    G = nx.DiGraph()
    for i, vecinos in enumerate(grafo):
        for j, peso in vecinos:
            G.add_edge(lista_nodos[i], lista_nodos[j], weight=peso)
    pos = nx.circular_layout(G) if tipo == "manual" else nx.spring_layout(G, seed=42)
    nx.draw_networkx_nodes(G, pos, node_size=800,
                           node_color="skyblue" if tipo == "manual" else "lightgreen")
    nx.draw_networkx_labels(G, pos, font_weight="bold")
    for u, v in G.edges():
        if G.has_edge(v, u) and u < v:
            nx.draw_networkx_edges(G, pos, edgelist=[(u, v)],
                                   connectionstyle='arc3,rad=0.25',
                                   arrows=True, arrowsize=20, edge_color='gray')
            nx.draw_networkx_edges(G, pos, edgelist=[(v, u)],
                                   connectionstyle='arc3,rad=-0.25',
                                   arrows=True, arrowsize=20, edge_color='gray')
        elif not G.has_edge(v, u):
            nx.draw_networkx_edges(G, pos, edgelist=[(u, v)],
                                   connectionstyle='arc3,rad=0.0',
                                   arrows=True, arrowsize=20, edge_color='gray')
    seen_pairs = set()
    for u, v, data in G.edges(data=True):
        peso = data['weight']
        x1, y1 = pos[u]
        x2, y2 = pos[v]
        xm, ym = (x1 + x2) / 2, (y1 + y2) / 2
        if G.has_edge(v, u):
            if (v, u) not in seen_pairs:
                plt.text(xm, ym + 0.1, str(peso), fontsize=9, color='black',
                         bbox=dict(facecolor='white', edgecolor='none', boxstyle='round,pad=0.2'))
            else:
                plt.text(xm, ym - 0.1, str(peso), fontsize=9, color='black',
                         bbox=dict(facecolor='white', edgecolor='none', boxstyle='round,pad=0.2'))
            seen_pairs.add((u, v))
            seen_pairs.add((v, u))
        else:
            plt.text(xm, ym, str(peso), fontsize=9, color='black',
                     bbox=dict(facecolor='white', edgecolor='none', boxstyle='round,pad=0.2'))
    plt.axis('off')
    plt.show()
```

Nota. Captura de pantalla del código de la función.

Figura 10



Nota. Dibujo del grafo generado a partir de la matriz de adyacencia.

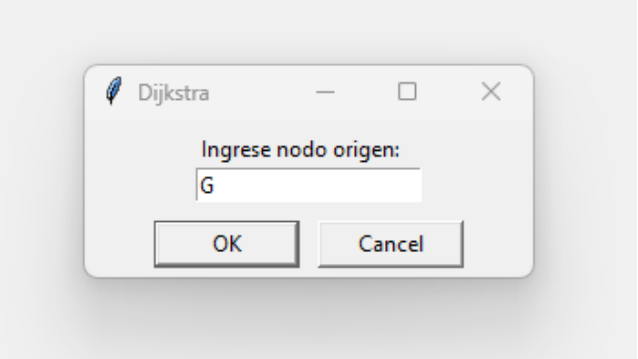
- **Algoritmo de Dijkstra:** Utilizando el algoritmo de Dijkstra el programa mostrará para encontrar la ruta más corta de un nodo a otro.

Figura 11

```python
def dijkstra_detallado(lista_nodos, grafo, origen, destino, text_widget):
    n = len(lista_nodos)
    dist = {v: float('inf') for v in lista_nodos}
    previo = {v: [] for v in lista_nodos}
    visitado = {v: False for v in lista_nodos}
    dist[origen] = 0
    for _ in range(n):
        no_visitados = {v: dist[v] for v in lista_nodos if not visitado[v]}
        if not no_visitados:
            break
        actual = min(no_visitados, key=no_visitados.get)
        visitado[actual] = True
        text_widget.insert(tk.END, f"\nVisitando nodo: {actual}")
        i = lista_nodos.index(actual)
        for j, peso in grafo[i]:
            vecino = lista_nodos[j]
            if dist[actual] + peso < dist[vecino]:
                dist[vecino] = dist[actual] + peso
                previo[vecino] = [actual]
            elif dist[actual] + peso == dist[vecino]:
                previo[vecino].append(actual)
        text_widget.insert(tk.END, f"\nDistancias actuales:\n")
        for v in lista_nodos:
            d = dist[v] if dist[v] != float('inf') else " "
            text_widget.insert(tk.END, f" {v}: {d} |")
        text_widget.insert(tk.END, "\n")
        text_widget.update()
    caminos = []
    def construir_caminos(nodo, camino):
        if nodo == origen:
            caminos.append([origen] + camino)
        else:
            for ant in previo[nodo]:
                construir_caminos(ant, [nodo] + camino)
    construir_caminos(destino, [])
    text_widget.insert(tk.END, f"\nDistancia mínima hasta {destino} = {dist[destino]}\n")
    text_widget.insert(tk.END, "Caminos mínimos posibles:\n")
    for c in caminos:
        text_widget.insert(tk.END, " => ".join(c) + "\n")
    text_widget.update()
    return dist[destino], caminos
```

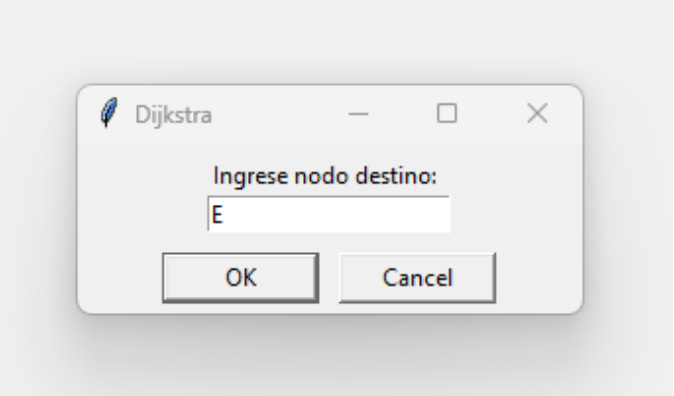
Nota. Captura de pantalla del código de la función que ejecuta el algoritmo de Dijkstra.

Figura 12



Nota. Se le pide seleccionar al usuario el nodo origen

Figura 13



Nota. Se le pide seleccionar al usuario el nodo destino.

Figura 14

```
Visitando nodo: G
Distancias actuales:
 A:    | B: 1 | C: 7 | D:    | E:    | F:    | G: 0 | H:    |

Visitando nodo: B
Distancias actuales:
 A:    | B: 1 | C: 5 | D:    | E: 8 | F: 10 | G: 0 | H:    |

Visitando nodo: C
Distancias actuales:
 A: 6 | B: 1 | C: 5 | D:    | E: 8 | F: 8 | G: 0 | H:    |

Visitando nodo: A
Distancias actuales:
 A: 6 | B: 1 | C: 5 | D:    | E: 8 | F: 8 | G: 0 | H:    |

Visitando nodo: E
Distancias actuales:
 A: 6 | B: 1 | C: 5 | D: 10 | E: 8 | F: 8 | G: 0 | H:    |
```

```
Visitando nodo: F
Distancias actuales:
 A: 6 | B: 1 | C: 5 | D: 10 | E: 8 | F: 8 | G: 0 | H:    |

Visitando nodo: D
Distancias actuales:
 A: 6 | B: 1 | C: 5 | D: 10 | E: 8 | F: 8 | G: 0 | H:    |

Visitando nodo: H
Distancias actuales:
 A: 6 | B: 1 | C: 5 | D: 10 | E: 8 | F: 8 | G: 0 | H:    |

Distancia mínima hasta E = 8
Caminos mínimos posibles:
G => B => E
```

Nota. Se muestra la distancia mínima entre los nodos y los caminos mínimos posibles.

- **Menú principal:** Se tiene un menú principal en el cual se llama a todas las funciones mediante el uso de botones.

Figura 15

```python
class Proyecto:
    def __init__(self, root):
        self.root = root
        self.root.title("Problema del camino minimo - Dijkstra")
        self.lista_nodos = []
        self.grafo = []
        frame = ttk.Frame(root, padding=20)
        frame.grid(row=0, column=0, padx=10, pady=10)

        btn_config = {"font": ("Times New Roman", 11, "bold"), "width": 20, "height": 2}
        tk.Button(frame, text="Crear Grafo Aleatorio", command=self.crear_aleatorio, bg="#D0A798", **btn_config).grid(row=0, column=0, padx=10, pady=10, sticky="ew")
        tk.Button(frame, text="Crear Grafo Manual", command=self.crear_manual, bg="#D0A798", **btn_config).grid(row=0, column=1, padx=10, pady=10, sticky="ew")
        tk.Button(frame, text="Mostrar Matriz de Adyacencia", command=self.ver_matriz, bg="#D0A798", **btn_config).grid(row=1, column=0, columnspan=2, padx=10, pady=10, sticky="ew")
        tk.Button(frame, text="Dibujar Grafo Manual", command=self.dibujar_manual, bg="#D0A798", **btn_config).grid(row=2, column=0, padx=10, pady=10, sticky="ew")
        tk.Button(frame, text="Dibujar Grafo Aleatorio", command=self.dibujar_aleatorio, bg="#D0A798", **btn_config).grid(row=2, column=1, padx=10, pady=10, sticky="ew")
        tk.Button(frame, text="Ejecutar Dijkstra", command=self.ejecutar_dijkstra, bg="#D0A798", **btn_config).grid(row=3, column=0, padx=10, pady=10, sticky="ew")
        tk.Button(frame, text="Salir del programa", command=self.root.quit, bg="#D0A798", **btn_config).grid(row=3, column=1, padx=10, pady=10, sticky="ew")
        for i in range(4):
            frame.rowconfigure(i, weight=1)
        for j in range(2):
            frame.columnconfigure(j, weight=1)
        frame_text = tk.Frame(root)
        frame_text.grid(row=4, column=0, padx=10, pady=10, sticky="nsew")
        self.text = tk.Text(frame_text, width=80, height=20)
        self.text.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
        scrollbar = tk.Scrollbar(frame_text, command=self.text.yview)
        scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
        self.text.config(yscrollcommand=scrollbar.set)
```

```python
def crear_aleatorio(self):
    try:
        n = simpledialog.askinteger("Grafo Aleatorio", "Ingrese el número de nodos (8-16):")
        if n is None:
            return
        if n < 8 or n > 16:
            raise ValueError
        self.lista_nodos, self.grafo = crear_grafoAleatorio(n)
        messagebox.showinfo("Éxito", "Grafo aleatorio generado correctamente.")
    except:
        messagebox.showerror("Error", "Ingrese un número válido entre 8 y 16.")

def crear_manual(self):
    try:
        n = simpledialog.askinteger("Grafo Manual", "Ingrese el número de nodos (3-10):")
        if n is None:
            return
        if n < 3 or n > 10:
            raise ValueError
        self.text.delete("1.0", tk.END)
        self.lista_nodos, self.grafo = crear_grafoManual(n, self.text)
        messagebox.showinfo("Éxito", "Grafo manual creado correctamente.")
    except:
        messagebox.showerror("Error", "Ingrese un número válido entre 3 y 10.")
```

```python
def ver_matriz(self):
    if not self.grafo:
        messagebox.showwarning("Aviso", "Primero cree un grafo.")
        return
    texto = mostrar_matriz(self.lista_nodos, self.grafo)
    self.text.delete("1.0", tk.END)
    self.text.insert(tk.END, texto)

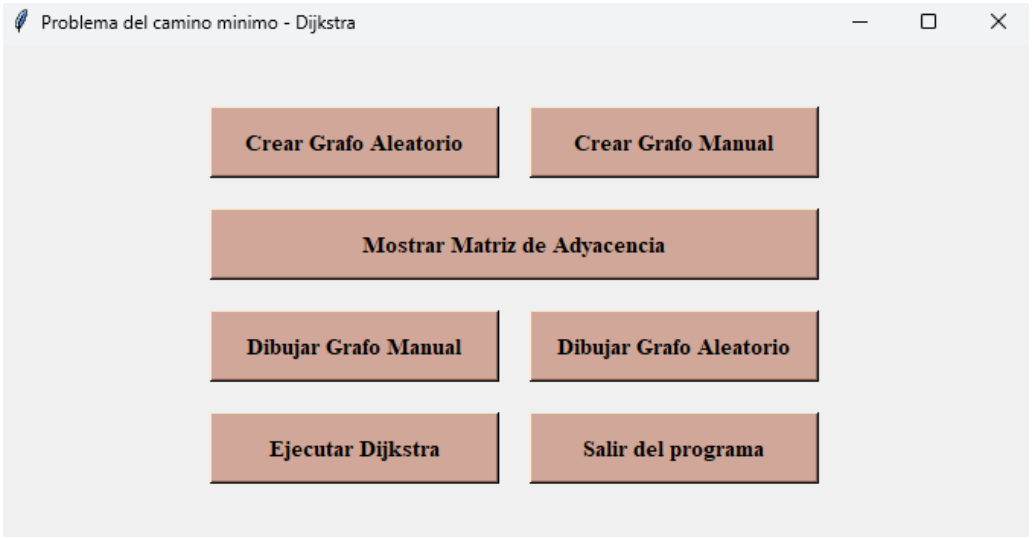
def dibujar_manual(self):
    if not self.grafo:
        messagebox.showwarning("Aviso", "Primero cree un grafo.")
        return
    dibujar_grafo(self.lista_nodos, self.grafo, "manual")

def dibujar_aleatorio(self):
    if not self.grafo:
        messagebox.showwarning("Aviso", "Primero cree un grafo.")
        return
    dibujar_grafo(self.lista_nodos, self.grafo, "aleatorio")
```

```python
def ejecutar_dijkstra(self):
    if not self.grafo:
        messagebox.showwarning("Aviso", "Primero cree un grafo.")
        return
    origen = simpledialog.askstring("Dijkstra", "Ingrese nodo origen:").upper()
    destino = simpledialog.askstring("Dijkstra", "Ingrese nodo destino:").upper()
    if origen is None or destino is None:
        return
    if origen not in self.lista_nodos or destino not in self.lista_nodos:
        messagebox.showerror("Error", "Nodos inválidos.")
        return
    self.text.delete("1.0", tk.END)
    dist, caminos = dijkstra_detallado(self.lista_nodos, self.grafo, origen, destino, self.text)
```

Nota. Se muestra el código de la función encargada de crear el menú principal y toda la interfaz.

Figura 16



Nota. Se muestra el diseño del menú principal.

Figura 17

```
if __name__ == "__main__":
    root = tk.Tk()
    proyecto = Proyecto(root)
    root.mainloop()
```

Nota. Código principal que ejecuta el programa.

7. Conclusiones

- **Importancia del Algoritmo de Dijkstra en la Resolución de Problemas de Optimización**
  El proyecto muestra cómo el algoritmo de Dijkstra es fundamental para resolver el problema del camino mínimo en grafos ponderados. Su eficiencia y aplicabilidad en contextos tan diversos como redes de transporte, telecomunicaciones, y navegación GPS evidencian su relevancia en la optimización de recursos y tiempos en escenarios reales. A través de este algoritmo, se pueden determinar las rutas más cortas de manera sistemática y confiable, lo que lo convierte en una herramienta esencial en muchos ámbitos de la tecnología moderna.

- **Aplicaciones del Problema del Camino Mínimo**
  Las aplicaciones prácticas del problema del camino mínimo y el algoritmo de Dijkstra son vastas y abarcan diversas áreas como el análisis de redes de transporte, la

optimización de sistemas logísticos, la gestión de redes de telecomunicaciones y la planificación de rutas en GPS. Esto subraya la importancia de este concepto en la vida cotidiana, pues está directamente vinculado con la eficiencia de los sistemas que usamos a diario, desde la navegación en vehículos hasta la transmisión de datos a través de internet.

- **Desarrollo de un Proyecto Colaborativo y Multidisciplinario**
  El trabajo en equipo en el desarrollo de este proyecto fue clave para lograr un avance del 50% en la implementación del algoritmo de Dijkstra. Cada miembro del equipo asumió roles bien definidos, trabajando tanto en la parte teórica como en la programación práctica. La cooperación y el uso de herramientas como Google Docs y reuniones virtuales facilitaron el flujo constante de ideas, permitiendo que el proyecto avanzara de manera eficiente y ordenada. Esta experiencia demuestra la importancia de la colaboración interdisciplinaria para abordar proyectos complejos.

- **Impacto del Uso de Herramientas de Visualización**
  La implementación de herramientas gráficas y de visualización, como la representación de la matriz de adyacencia y la graficación del grafo, proporcionó un valor agregado al proyecto. Estas herramientas no solo mejoran la comprensión del algoritmo de Dijkstra, sino que también hacen que el proceso de resolución de problemas sea más accesible, interactivo y visualmente intuitivo. Esto facilita el aprendizaje y la resolución de problemas complejos, especialmente en entornos educativos y de investigación.

- **Perspectiva de Continuidad y Mejora en el Proyecto**
  Aunque se ha alcanzado un avance significativo en la primera fase del proyecto, el código y las funcionalidades de la herramienta están diseñados para ser mejorados y ampliados en futuras entregas. Este avance inicial sienta las bases para la implementación completa de un sistema capaz de manejar diversos tipos de grafos y problemas de optimización. La incorporación de más funcionalidades, como la interfaz de usuario y la posibilidad de gestionar diferentes tipos de grafos, ampliará las capacidades del programa y permitirá su uso en una gama más amplia de aplicaciones.

- **Reflexión sobre el Aprendizaje y la Investigación**
  La realización de este proyecto no solo permitió el aprendizaje profundo de conceptos clave en teoría de grafos y algoritmos de optimización, sino que también reforzó el entendimiento de cómo la matematización de problemas reales, como el camino mínimo, puede transformar sectores completos. A través de la implementación práctica del algoritmo y la interacción con herramientas computacionales, los integrantes del equipo consolidaron tanto sus conocimientos teóricos como sus habilidades de programación, lo que resulta esencial para su formación profesional en ingeniería y ciencias computacionales.

8. Bibliografía

- Chacón, J. L. (2005). *Introducción a la Teoría de Grafos.* Alfaomega. https://libroweb.alfaomega.com.mx/book/685/free/ovas_statics/cap7/Introduccion%20a%20la%20teoria%20de%20grafos.%20Chacon,%20Jose%20Luis..pdf

- Cooke, D., Halsey, E., & Santucci, D. (2003). *The shortest path problem: A historical perspective. Interfaces, 33*(4), 1–13. https://www.scirp.org/reference/referencespapers?referenceid=1317535&utm_source=chatgpt.com

- Cornell University. (2001)**.** *Edsger W. Dijkstra: Shortest path algorithm.* Cornell University Computer Science Department. Recuperado de https://www.cs.cornell.edu/courses/JavaAndDS/shortestPath/01shortestPathHistory.pdf?utm_source=chatgpt.com

- García H., S. (2019). *Búsqueda de comunidades en grafos ponderados. Detección de tramas de blanqueo de capitales.* [Tesis de Maestría, Universidad Politécnica de Madrid]. Archivo Digital UPM. https://oa.upm.es/62942/

- Lluch, J. (2010). *Representación de grafos. Matriz de adyacencia*. Universitat Politècnica de València. Repositorio institucional RiuNet. https://riunet.upv.es/entities/publication/ec3166a1-dda5-48e1-b3d2-f3d75f7b13a1

- Martínez, O. (s.f.). *Introducción a la teoría de grafos y sus aplicaciones en GEPHI.* https://math.uniandes.edu.co/~o.martinez25/files/otros/introGrafos.pdf

- Muñóz E., B. (2010). *Análisis del problema del camino mínimo en redes urbanas.* [Tesis de Especialidad, Universitat Politècnica de Catalunya]. UPCommons. https://upcommons.upc.edu/entities/publication/08a6bd0d-1c2f-4572-8890-312a4d2c3f79

- Sánchez T., G. & Lozano T., V. (2001). *Algoritmo de Dijkstra. Un tutorial interactivo. VII Jornadas de Enseñanza Universitaria de la Informática (JENUI 2001),* ISBN 84-7632-657-2, pp. 254-258. https://rua.ua.es/entities/publication/edc40560-5e6b-4ada-973c-8c313ef75c4d